7장. FastAPI 고급 기능

본 장에서는 실제 웹 서비스를 구축할 때 반드시 활용하게 되는 FastAPI의 고급 컴포넌트들을 학습합니다. 라이프사이클을 제어하는 **Lifespan** 핸들러, 핵심 유연성을 담당하는 **의존성 주입(DI)**, 연산을 백그라운드로 넘겨 응답을 고속화하는 **BackgroundTasks**, 그리고 멀티파트 **파일 처리** 기법을 다룹니다.

7.1 Lifespan: 서버 실행과 종료 시점 제어

FastAPI는 앱이 구동될 때 데이터베이스 커넥션 풀을 미리 초기화하고, 서버가 종료될 때 커넥션을 정상적으로 닫아주는 수명 주기 제어 기능을 지원합니다. 옛날 방식인 `@app.on_event("startup")` 대신 최신 Python 표준인 `contextmanager` 기반의 **Lifespan** 패턴을 권장합니다.

```
from fastapi import FastAPI
from contextlib import asynccontextmanager

# 💉 데이터베이스 커넥션풀 초기 가상 함수 예시
async def init_db():
    print("🔋 [Lifespan] 데이터베이스 연결 활성화 완료.")

async def shutdown_db():
    print("🪫 [Lifespan] 데이터베이스 세션 풀 안전 차단 완료.")

# 애플리케이션 수명 주기 관리 매니저 정의
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 1. startup (서버 구동 시작 시 실행)
    await init_db()
    yield # --- 이 지점에서 대기하다가 앱 동작 수행 ---
    # 2. shutdown (서버 종료 시 실행)
    await shutdown_db()

# app 정의에 lifespan 인자 등록
app = FastAPI(lifespan=lifespan)
```

7.2 의존성 주입(Dependency Injection) 심화

FastAPI는 `Depends` 키워드를 활용해 강력한 의존성 주입 체계를 제공합니다. 의존성 주입을 활용하면 코드 중복을 제거할 뿐 아니라 데이터베이스 연결 관리, 보안 검증, 세션 상태 유지를 정교하게 처리할 수 있습니다.

```
from fastapi import Depends

# 공통 쿼리 파라미터 클래스 예제
class CommonQueryParams:
    def __init__(self, q: str | None = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

@app.get("/items")
def read_items(commons: CommonQueryParams = Depends()):
    # Depends에 주입하고자 하는 클래스를 명시하면 파라미터가 자동으로 매핑 및 전달됩니다.
    return {"q": commons.q, "skip": commons.skip, "limit": commons.limit}
```

7.3 백그라운드 태스크(BackgroundTasks)

사용자에게 빠른 응답을 전달해야 하고, 백그라운드에서 오랜 시간이 소요되는 무거운 로직(예: 메일 발송, 분석 파일 생성, 시스템 모니터링 등)을 수행해야 하는 경우, BackgroundTasks를 이용해 비동기로 처리를 지연 실행시킬 수 있습니다.

```
from fastapi import BackgroundTasks

def send_welcome_email(email: str):
    # 실제 메일 서버 발송 로직이 유입되는 부분 (임시 3초 딜레이 가정)
    import time
    time.sleep(3)
    print(f" [Background] {email} 주소로 가입 환영 메일을 발송 완료했습니다.")

@app.post("/register")
def register_user(email: str, background_tasks: BackgroundTasks):
    """회원가입 요청 시 즉시 응답을 전송하고 메일 발송은 백그라운드에 위임합니다."""
    # 백그라운드 태스크 대기열에 작업 등록
    background_tasks.add_task(send_welcome_email, email)
    return {"message": "가입 승인이 완료되었습니다. 환영 이메일은 곧 발송됩니다."}
```

7.4 파일 업로드 및 다운로드 (Multipart / UploadFile)

사용자가 보낸 이미지나 문서 파일을 서버에 저장하고, 저장된 파일을 클라이언트가 안전하게 내려받도록 경로(Response)를 구성하는 방법을 설명합니다.

```python
import os
import shutil
from fastapi import UploadFile, File, HTTPException
from fastapi.responses import FileResponse

# 파일이 적재될 임시 폴더
UPLOAD_DIR = "./uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

@app.post("/upload")
def upload_file(file: UploadFile = File(...)) -> dict:
    """UploadFile 객체를 통해 파일명과 메타데이터에 쉽게 접근하고 안전하게 저장합니다."""
    file_path = os.path.join(UPLOAD_DIR, file.filename)

    # 스트림 형태로 파일을 받아서 저장 (대용량 파일에 메모리 과부하가 걸리지 않는 안전한 방식)
    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    return {"filename": file.filename, "size": file.size, "status": "Uploaded Successfully"}

@app.get("/download/{filename}")
def download_file(filename: str) -> FileResponse:
    """서버의 지정된 경로에서 파일을 로드하여 클라이언트 브라우저로 내려보냅니다."""
    file_path = os.path.join(UPLOAD_DIR, filename)
    if not os.path.exists(file_path):
        raise HTTPException(status_code=404, detail="지정된 파일을 찾을 수 없습니다.")

    return FileResponse(
        path=file_path,
        media_type="application/octet-stream", # 범용 이진 파일 스트림 헤더
        filename=filename # 브라우저가 저장할 기본 파일명 지정
    )
```